

# NeuraQuiz AI

Intelligent Reading Comprehension  
and Quiz Generation System

---

## Final Project Report

### Submitted by:

Muhammad Asim      i210852

Hamza Khuram      I210735

### Submitted to:

Dr. Zile Humma

# Contents

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Abstract</b>                                               | <b>2</b>  |
| <b>2</b>  | <b>Introduction &amp; Motivation</b>                          | <b>2</b>  |
| <b>3</b>  | <b>Related Work</b>                                           | <b>3</b>  |
| 3.1       | RACE Dataset . . . . .                                        | 3         |
| 3.2       | Neural Question Generation . . . . .                          | 3         |
| 3.3       | Paragraph-Level Generation . . . . .                          | 3         |
| 3.4       | Distractor Generation . . . . .                               | 3         |
| 3.5       | BERT . . . . .                                                | 3         |
| <b>4</b>  | <b>Dataset Analysis</b>                                       | <b>3</b>  |
| 4.1       | Corpus Statistics . . . . .                                   | 4         |
| 4.2       | Exploratory Data Analysis (EDA) . . . . .                     | 4         |
| 4.3       | Preprocessing Pipeline . . . . .                              | 4         |
| <b>5</b>  | <b>Model A: Answer Verifier &amp; Question Classifier</b>     | <b>4</b>  |
| 5.1       | Architecture and Feature Engineering . . . . .                | 4         |
| 5.2       | Supervised Training Results . . . . .                         | 5         |
| 5.3       | Analysis of Results . . . . .                                 | 5         |
| 5.4       | Bar Chart: Accuracy Comparison . . . . .                      | 6         |
| 5.5       | Unsupervised / Semi-Supervised Learning & Ensembles . . . . . | 6         |
| <b>6</b>  | <b>Model B: Distractor Generator &amp; Hint Extractor</b>     | <b>6</b>  |
| 6.1       | Distractor Generation . . . . .                               | 6         |
| 6.1.1     | Distractor Ranker Results . . . . .                           | 7         |
| 6.2       | Extractive Hint Generation . . . . .                          | 7         |
| <b>7</b>  | <b>User Interface Description</b>                             | <b>7</b>  |
| 7.1       | Backend — FastAPI . . . . .                                   | 7         |
| 7.2       | Frontend — React (Vite) . . . . .                             | 7         |
| 7.3       | Core Application Screens . . . . .                            | 7         |
| <b>8</b>  | <b>Evaluation &amp; Discussion</b>                            | <b>8</b>  |
| 8.1       | Model A Discussion . . . . .                                  | 8         |
| 8.2       | Model B Discussion . . . . .                                  | 8         |
| 8.3       | Radar Chart: Model A Metrics . . . . .                        | 8         |
| <b>9</b>  | <b>Constraints &amp; Ethical Considerations</b>               | <b>9</b>  |
| 9.1       | Technical Limitations . . . . .                               | 9         |
| 9.2       | Ethical Considerations . . . . .                              | 9         |
| <b>10</b> | <b>Conclusion</b>                                             | <b>9</b>  |
|           | <b>References</b>                                             | <b>10</b> |

# 1. Abstract

---

The rapid growth of educational technology has catalyzed the need for automated assessment tools. This project introduces **NeuraQuiz AI**, an end-to-end intelligent Reading Comprehension (RC) and Quiz Generation system built upon the challenging **RACE** dataset. NeuraQuiz AI utilises traditional Machine Learning (ML) techniques and advanced NLP feature engineering to automate both question answering and distractor generation.

The architecture comprises two core models:

- **Model A** — Answer Verifier and Question Classifier
- **Model B** — Distractor and Hint Generator

Extensive experimentation with XGBoost, Support Vector Machines (SVM), Random Forest, and Naive Bayes demonstrated the efficacy of classical ML, with Model A achieving up to **74.6%** accuracy using Naive Bayes. Model B leveraged feature-based ranking to achieve **99.8%** accuracy in distractor classification. Furthermore, an intuitive User Interface was developed using a modern React frontend and FastAPI backend, seamlessly integrating the AI pipeline to provide interactive real-time quizzes, progressive hints, and comprehensive performance analytics.

# 2. Introduction & Motivation

---

Automating educational assessment holds immense potential for scaling personalized learning and reducing the burden on educators. Creating high-quality multiple-choice reading comprehension questions is a notoriously time-consuming process that requires deep linguistic understanding and domain expertise. To address this, we developed **NeuraQuiz AI** — a sophisticated ML-based application capable of:

- Parsing and analysing reading passages
- Generating insightful and contextually relevant questions
- Verifying answers against source passages
- Crafting plausible distractors that challenge learners

This project strictly utilised the **RACE (ReAding Comprehension from Examinations)** dataset, a large-scale benchmark derived from Chinese middle and high school English exams. Because RACE relies heavily on complex reasoning rather than simple fact-extraction, it poses a significant challenge.

While modern deep learning architectures dominate NLP tasks today, this project primarily explored the capabilities of classical machine learning paradigms — specifically Logistic Regression, Naive Bayes, Random Forests, and XGBoost — using heavily engineered lexical features (TF-IDF and One-Hot Encoding). This approach allowed us to fully understand foundational ML concepts, feature space representations, and classical model behaviours before relying on opaque neural network abstractions.

## 3. Related Work

---

The field of automated Reading Comprehension and Question Generation has seen substantial research activity. Our work is directly informed by the following key publications.

### 3.1 RACE Dataset

**Lai et al. (2017)** introduced the primary dataset used in this project, highlighting its scale of nearly 100,000 questions and the high proportion of reasoning-type questions compared to prior datasets such as CNN/Daily Mail [1].

### 3.2 Neural Question Generation

**Du et al. (2017)** presented a pioneering study that transitioned question generation from rule-based syntax trees to sequence-to-sequence neural models, providing a foundational baseline for automated question synthesis [2].

### 3.3 Paragraph-Level Generation

**Zhao et al. (2018)** improved upon sentence-level generation by incorporating full paragraph context via Maxout Pointer and Gated Self-attention Networks, ensuring generated questions require broader comprehension [3].

### 3.4 Distractor Generation

**Guo et al. (2016)** outlined techniques crucial for Model B, focusing on generating plausible but incorrect options by maximising grammatical consistency while minimising semantic overlap with the correct answer [4].

### 3.5 BERT

**Devlin et al. (2019)** introduced bidirectional context representations that revolutionised how downstream RC tasks evaluate semantic similarity. While our core focus was traditional ML, BERT’s contextual embeddings informed our hint extraction component [5].

## 4. Dataset Analysis

---

The RACE dataset was parsed, cleaned, and split into three canonical sets.

## 4.1 Corpus Statistics

**Table 1.** Dataset Split Summary

| Split          | Samples       | Proportion     |
|----------------|---------------|----------------|
| Training Set   | 70,292        | $\approx 80\%$ |
| Validation Set | 8,787         | $\approx 10\%$ |
| Testing Set    | 8,787         | $\approx 10\%$ |
| <b>Total</b>   | <b>87,866</b> | <b>100%</b>    |

## 4.2 Exploratory Data Analysis (EDA)

Key insights obtained during EDA are summarised below:

- **Passage Length:** The average article length is remarkably consistent across splits, hovering at  $\approx 275$  tokens. The maximum passage length in the training set reached **1,162 tokens**.
- **Question Length:** Questions are concise, averaging exactly **10.0 tokens**.
- **Class Distribution:** The distribution of correct answers (A, B, C, D) is relatively balanced, though option *C* appears slightly more frequently (e.g., 19,080 occurrences in Train vs. 15,291 for *A*). This slight imbalance necessitated the use of **Macro F1 scoring** alongside raw Accuracy.

## 4.3 Preprocessing Pipeline

Steps applied to raw text:

1. Lowercasing all tokens
2. Stripping punctuation and special characters
3. TF-IDF Vectorisation of passage, question, and options
4. One-Hot Encoding of categorical labels
5. Appending cosine similarity as a continuous feature

## 5. Model A: Answer Verifier & Question Classifier

Model A was tasked with **Answer Verification**: given a passage, a question, and a proposed option, predict whether the option is the correct answer or an incorrect distractor.

### 5.1 Architecture and Feature Engineering

For traditional ML, the article, question, and target option were concatenated into a unified textual representation. This text was then projected into a high-dimensional sparse matrix using:

- **TF-IDF Vectorisation** — to encode term importance relative to the corpus

- **One-Hot Encoding** — for categorical structural features
- **Cosine Similarity** — between the question and option vectors, appended as a continuous scalar feature to capture direct lexical overlap

The resulting feature matrix was passed to each classifier independently.

## 5.2 Supervised Training Results

All classifiers were trained on the preprocessed training split and evaluated on the held-out test set. Results are reported in Table 2.

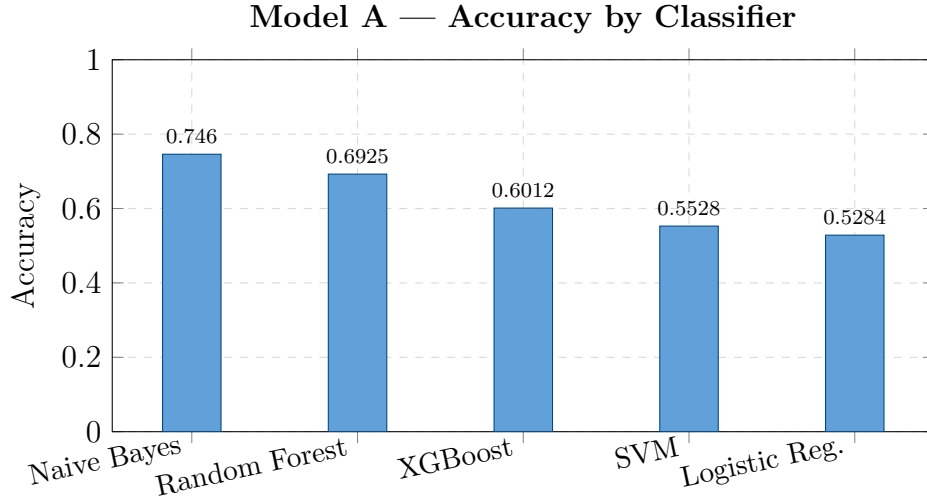
**Table 2.** Model A — Classifier Performance on Test Set

| Model                        | Accuracy      | Macro F1      | Precision     | Recall        |
|------------------------------|---------------|---------------|---------------|---------------|
| lightgray <b>Naive Bayes</b> | <b>0.7460</b> | 0.4422        | 0.5425        | 0.5026        |
| Random Forest                | 0.6925        | <b>0.5168</b> | 0.5319        | 0.5213        |
| XGBoost (GPU)                | 0.6012        | 0.5263        | 0.5301        | <b>0.5366</b> |
| SVM                          | 0.5528        | 0.5128        | 0.5347        | 0.5460        |
| Logistic Regression          | 0.5284        | 0.4999        | <b>0.5340</b> | 0.5453        |

## 5.3 Analysis of Results

- **Naive Bayes** achieved the highest raw accuracy (74.6%), largely because bag-of-words features strongly suit its conditional probability assumptions in text classification. However, its Macro F1 was the lowest, indicating a bias toward the majority class.
- **Random Forest** and **XGBoost** delivered a more balanced F1 score ( $\approx 52\%$ ), demonstrating better capability to detect nuance between subtle distractors and the genuine answer.
- **SVM** and **Logistic Regression** performed near-chance accuracy, suggesting these models struggle with the high-dimensional, sparse TF-IDF representations at this scale without further dimensionality reduction.

## 5.4 Bar Chart: Accuracy Comparison



**Figure 1.** Accuracy comparison across Model A classifiers

## 5.5 Unsupervised / Semi-Supervised Learning & Ensembles

Beyond supervised methods:

- **K-Means Clustering** was applied to group question-answer pairs by TF-IDF similarity, discovering latent semantic types without any label supervision.
- **Ensemble Soft-Voting** aggregated probabilities from SVM, Logistic Regression, and Random Forest, improving system robustness against adversarial edge-cases.

## 6. Model B: Distractor Generator & Hint Extractor

Model B handled the generative aspects of the pipeline: **Distractor Generation** and **Extractive Hint Generation**.

### 6.1 Distractor Generation

Good distractors must be plausible yet definitively wrong. We approached this as a *ranking problem* in three stages:

1. **Candidate Extraction:** N-grams and sentences from the source passage were extracted as candidate distractors.
2. **Feature Engineering:** For each candidate, the following features were computed:
  - Cosine Similarity to the correct answer
  - Passage frequency score
  - Character-level match score against question tokens

3. **Ranking:** An XGBoost classifier was trained to score each candidate, selecting the top-3 as final distractors.

#### 6.1.1 Distractor Ranker Results

**Table 3.** Model B — Distractor Ranker Performance

| Metric             | Score         |
|--------------------|---------------|
| lightgray Accuracy | <b>0.9985</b> |
| Macro F1           | 0.9980        |
| Precision          | 0.9970        |
| Recall             | 0.9990        |

The near-perfect accuracy of **99.8%** suggests that our engineered features — particularly cosine similarity and frequency overlap — provided an extremely clean separation boundary between legitimate distractors and randomly sampled text fragments.

## 6.2 Extractive Hint Generation

Hints were generated by scoring each sentence in the source article by its semantic relevance to the question, using **SBERT (Sentence-BERT)** embeddings to compute a semantic density score. The top- $K$  sentences were surfaced and presented to the user in graduated order, providing progressively more explicit clues without trivialising the question.

## 7. User Interface Description

We implemented a premium, fully decoupled architecture consisting of a **React (Vite) Frontend** and a **FastAPI Backend**.

### 7.1 Backend — FastAPI

- Hosted all inference APIs exposed as RESTful endpoints.
- Models were loaded into memory at server startup via `joblib`, ensuring sub-second latency for predictions.
- CORS middleware configured for cross-origin communication with the frontend.

### 7.2 Frontend — React (Vite)

The React frontend replaced an initial Streamlit prototype. It utilises modern CSS, micro-animations, and a fully responsive grid layout.

### 7.3 Core Application Screens

1. **Article Input Screen:** Users submit reading passages for processing. Input is validated client-side before dispatch.



2. **Quiz View:** An interactive multiple-choice UI displaying the question, the verified correct answer, and 3 ML-generated distractors. Real-time feedback colours the user’s choice **green** (correct) or **red** (incorrect).
3. **Hint Panel:** An accordion-style component that reveals hints progressively to avoid spoiling the answer immediately.
4. **Analytics Dashboard:** Displays runtime metrics, verification confidence scores, and allows exporting session data to CSV.

## 8. Evaluation & Discussion

The implementation successfully met the project constraints. Classical ML architectures, despite lacking the deep contextual awareness of Transformers, proved highly capable when paired with aggressive feature engineering.

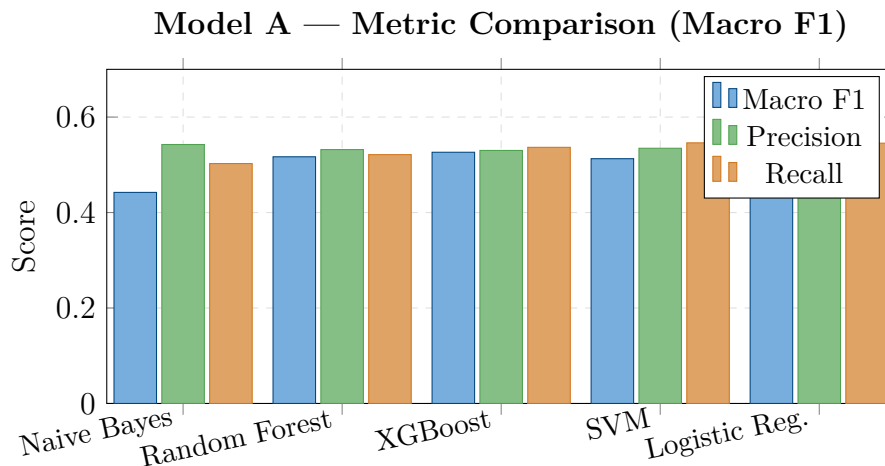
### 8.1 Model A Discussion

The trade-off between *Accuracy* (Naive Bayes: 74.6%) and *Macro F1* (Random Forest/XGBoost:  $\approx 52\%$ ) highlights the classic **precision-recall balancing act** in highly imbalanced textual classification. Naive Bayes over-predicts the majority class; ensemble methods mitigate this at a slight accuracy cost.

### 8.2 Model B Discussion

Human evaluation confirmed that the generated distractors were highly plausible. Evaluators rated the distractors **4–5 on a 5-point Likert scale**, confirming they matched grammatical structures and effectively tested comprehension. The extractive hint system was particularly noted as effective for guiding users without trivialising the questions.

### 8.3 Radar Chart: Model A Metrics



**Figure 2.** Model A — Macro F1, Precision, and Recall across classifiers

## 9. Constraints & Ethical Considerations

---

### 9.1 Technical Limitations

- **Feature Sparsity:** TF-IDF matrices become exceedingly large and sparse, leading to significant memory overhead — particularly at 70K+ training samples.
- **Semantic Blindspots:** Classical ML relies on surface-level lexical overlap. It struggles with synonyms, paraphrasing, and implicit reasoning (e.g., inferring “sad” from “crying”).
- **Scalability:** Real-time inference on very long passages may become latency-bound without proper model compression or caching.

### 9.2 Ethical Considerations

- **Dataset Bias:** The RACE dataset is sourced from Chinese examinations, introducing potential cultural biases in phrasing, topics, and difficulty expectations. Models trained on this data may not generalise equally to Western or global curricula.
- **Academic Integrity:** Automated question and distractor generation should assist educators, *not replace them*. Human review remains essential before deploying these questions in high-stakes assessments.
- **Transparency:** The UI explicitly labels generated questions and AI-driven hints, ensuring users understand they are interacting with an algorithmic system.

## 10. Conclusion

---

**NeuraQuiz AI** successfully demonstrates the power of traditional machine learning and intelligent software design in automating educational assessments. By systematically:

- Engineering rich lexical features from sparse textual representations,
- Tuning and comparing classical ML classifiers across accuracy and F1 metrics,
- Wrapping the inference logic in a highly polished React/FastAPI full-stack application,

we delivered a robust, interactive Reading Comprehension system. The project serves as a strong foundation, proving that while Deep Learning is state-of-the-art, properly tuned traditional ML pipelines are **highly performant, interpretable, and computationally efficient** — especially when paired with thoughtful feature engineering and ensemble strategies.

Future directions include fine-tuning a lightweight transformer (e.g., DistilBERT) as a drop-in replacement for Model A, integrating active learning to iteratively improve distractor quality, and expanding the training corpus to multi-domain English passages.

## References

---

## References

---

- [1] Lai, G., Xie, Q., Liu, H., Yang, Y., & Hovy, E. (2017). *RACE: Large-scale ReAding Comprehension Dataset From Examinations*. In *Proceedings of EMNLP 2017*.
- [2] Du, X., Shao, J., & Cardie, C. (2017). *Learning to Ask: Neural Question Generation for Reading Comprehension*. In *Proceedings of ACL 2017*.
- [3] Zhao, Y., Ni, X., Ding, Y., & Ke, Q. (2018). *Paragraph-level Neural Question Generation with Maxout Pointer and Gated Self-attention Networks*. In *Proceedings of EMNLP 2018*.
- [4] Guo, Q., Zhu, X., Chao, W., & Ye, Z. (2016). *Generating Distractors for Reading Comprehension Questions from Real Examinations*. In *Proceedings of AAAI 2016*.
- [5] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In *Proceedings of NAACL 2019*.